

GRD and the Other GRD: Goal Recognition Design and Guard-Rail Design

גבולות גזרה

Omer Belsky

June 2025

Abstract

Ensuring that Large Language Models (LLMs) comply with desired behavioral specifications, such as safety constraints, stylistic requirements, or task instructions, remains a key challenge due to their open-ended generative nature. We propose a novel compliance framework based on Goal Recognition Design (GRD), a classical-planning technique that reduces ambiguity over an agent’s goals by strategically modifying its environment.

By modeling token-level LLM generation as a goal recognition problem, we adapt GRD to guide decoding strategies toward trajectories that allow us to determine earlier in the generation process whether a model is compliant or not, allowing for early termination of generation.

1 Introduction

AG: add citations to support your claims LLMs are capable of generating human-like text and are now deployed across a wide range of applications. However, ensuring that their outputs consistently comply with ethical standards, safety constraints, domain-specific policies, or user-specified instructions remains a significant challenge. Compliance mechanisms—including but not limited to guardrails—aim to steer or constrain LLM behavior so that emerging generations remain aligned with these requirements.

In this work, we propose to view LLM generation as a sequential decision-making process and leverage tools from Goal Recognition Design (GRD) to detect non-compliant generation trajectories early. Our central insight is that undesirable or policy-violating completions can be modeled as goals within a planning framework, allowing GRD techniques to reduce ambiguity over such goals as generation unfolds. This enables earlier recognition of problematic trajectories and, consequently, earlier and more effective intervention. While safety guardrails provide a motivating example, the proposed formulation extends naturally to broader notions of LLM compliance.

2 Background

AG: related work involve works that tried to solve the same problem you do.
Here I just see background

2.1 Goal Recognition Design (GRD)

GRD, as formalized by Keren *et al.* [3], asks: given a planning domain and multiple goals, how can we modify the domain so that agents reveal their goal as early as possible? A key measure is the Worst Case Distinctiveness (WCD), the maximum number of actions an optimal agent can take before its goal is uniquely identified. By modifying the set of available actions (e.g., removing ambiguous paths), GRD reduces WCD and thus improves goal recognition speed.

2.2 How LLMs Work

AG: relevant citations? LLMs generate text by predicting one token at a time, typically using beam search or sampling-based decoding strategies like top- p or top- k . Each generation step can be viewed as selecting an action (token) from a probabilistic distribution conditioned on the input and previously generated tokens. The generation thus forms a trajectory in token space, with goals implicitly defined by the final text.

3 Formulation

We draw a conceptual analogy between grid-navigation in classical planning and token generation in LLMs. In both settings, an agent moves through a space (states of a grid or token prefixes) by taking actions (grid moves or selecting tokens).

We borrow the STRIPS formulation [3]. The design problem is represented as a tuple $D = \langle P, G \rangle$ where P is a planning domain $\langle F, I, A, C \rangle$, F a set of fluents, $I \subseteq F$ an initial state, A a set of actions, and C a non-negative cost function.

3.1 Grid-Navigation Formulation

In grid navigation, agents start from an initial square on a grid s_0 , and aim to reach one of several states on the grid defined as goals (G). The model designer modifies the environment (e.g., adds barriers) to disallow certain actions $a \in A$, making it easier to infer the agent’s intended goal earlier.

TODO: add images

3.2 Compliance Formulation

To define the design problem, we must first define a goal recognition problem for the LLM compliance setting. Every square on our grid is a possible generation by the LLM, making the set of states be the set of all possible permutations (repeating) of tokens in the LLM’s vocabulary. The action of taking a step on the grid can be equated to generating the next token, while the goal states of the LLM are either finishing a generation successfully or generating a non-compliant output. To relax the problem, we assume the agent (LLM) can deterministically choose from the set of top- p tokens.

We denote by V the vocabulary of the language model (LLM), and by L the maximum number of tokens the LLM can generate. Let

$$S = \{t_1 t_2 \dots t_k \mid t_i \in V, k \leq L\}$$

be the set of all possible token sequences of length at most L . For each partial generation $s \in S$, the model produces a vector of logits $\vec{l} = \text{LLM}(s)$, and we define

$$l_p(s) = \{t \in V \mid t \text{ is among the top-}p \text{ tokens according to } \vec{l}\}.$$

The set of fluents is given by

$$F = \{\text{llm-at}(s) \mid s \in S\} \cup \{\text{next}(s, s+t) \mid s \in S, t \in l_p(s)\},$$

where $s+t$ denotes the concatenation of sequence s with token t .

The initial state is defined as

$$I = \{\text{llm-at}(\epsilon)\},$$

where ϵ represents the empty string (i.e., the start of generation).

We define the set of actions as

$$A = \{ \langle \text{generate}(s, t), \text{pre: llm-at}(s), \text{eff: llm-at}(s+t) \wedge \neg \text{llm-at}(s) \rangle \mid s \in S, t \in l_p(s) \},$$

representing the operation of appending token t to the current generation s .

Finally, we allow for a cost function $C : A \rightarrow \mathbb{R}_{\geq 0}$, whose specific form is yet to be defined.

To define the set of goal states G , we must first formalize what constitutes a *non-compliant* generation. An example scenario can be the use of guardrails to ensure the LLM doesn’t generate harmful content. A common approach to implementing guardrails involves post-hoc evaluation using a separate LLM, as seen in LLAMA GUARD [2] and GUARDRAILS AI [1]. Zheng *et al.* [4] fine-tune a lightweight BERT model that achieves performance comparable to larger LLM-based evaluators.

We denote by

$$f : S \rightarrow [0, 1]$$

a classifier that assigns each generation $s \in S$ a probability of harmfulness. Given a predefined threshold $\tau \in [0, 1]$, we define the set of harmful goal states as

$$G_{\text{harm}} = \{s \in S \mid f(s) \geq \tau\}.$$

However, a generation may also terminate before producing harmful content. To capture this, we introduce a special terminal state g_{end} , which is reached deterministically (with probability 1) from any state that has appended the end-of-sequence token $\langle \text{EOS} \rangle$. Formally, if $s \in S$ ends with $\langle \text{EOS} \rangle$, then $\text{next}(s, g_{\text{end}})$ holds.

The complete set of goal states is therefore defined as

$$G = G_{\text{harm}} \cup \{g_{\text{end}}\}.$$

3.3 Guard-Rail Design Formulation

Following [3], the objective of the design problem is to minimize the worst-case distinctiveness (WCD), which measures the earliest point along an agent’s trajectory at which its intended goal becomes unambiguously identifiable under adversarial behavior. In the classical grid-walk setting, WCD can be computed directly because the state and action spaces are small: the agent has only a handful of possible moves, and the optimal paths to each goal state can be exhaustively evaluated. This allows one to identify, for every state, whether all optimal continuations lead to a single goal, thereby determining the moment at which the agent has effectively committed to that goal.

In the LLM generation setting, however, such an approach is infeasible. The vocabulary contains hundreds of thousands of tokens, the branching factor at each step is large, and the number of possible trajectories grows exponentially with sequence length. As a result, computing optimal paths or exact commitment points is prohibitively expensive. To address this, we introduce a surrogate notion of goal-directedness based on the harmfulness classifier f . The assumption is that trends in $f(s)$ reflect movement toward or away from non-compliant completions.

To approximate the commitment point required by WCD, we define two soft thresholds

$$\tau_{\text{compliant}} < \tau_{\text{noncompliant}} < \tau.$$

Crossing $\tau_{\text{noncompliant}}$ indicates that the agent is progressing toward a non-compliant (harmful) generation, while crossing $\tau_{\text{compliant}}$ indicates that the agent is progressing toward a compliant generation. These thresholds serve as practical analogues of commitment in settings where exact computation of optimal trajectories is not tractable.

Since very short partial generations may cause the classifier f to fluctuate and produce unreliable indications of directionality, we additionally impose a minimum-length requirement. A sequence must reach a length of at least k_{min} tokens before threshold crossings are considered meaningful, ensuring that WCD is evaluated only once sufficient linguistic context has accumulated.

References

- [1] Guardrails ai. <https://www.guardrailsai.com>.
- [2] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- [3] Sarah Keren, Avigdor Gal, and Erez Karpas. Goal recognition design. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 24, pages 154–162, 2014.
- [4] Aaron Zheng, Mansi Rana, and Andreas Stolcke. Lightweight safety guardrails using fine-tuned bert embeddings. *arXiv preprint arXiv:2411.14398*, 2024.